

Node.js & Express – Complete Theory Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 12 Oct 2026 – 18 Oct 2026 (Days 57–63)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm**, **3:00–5:30pm**, **6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Goal: Master Node.js internals and Express.js architecture for production backend development — event loop, middleware, REST design, validation, error handling, streams, security, and layered architecture.

Table of Contents

1. [What is Node.js?](#)
2. [Event Loop Deep Dive](#)
3. [Modules: CommonJS vs ESM](#)
4. [Express Architecture](#)
5. [Middleware Patterns](#)
6. [REST API Design](#)
7. [Validation & Sanitization](#)
8. [Error Handling](#)
9. [Streams & Buffers](#)
10. [Security Best Practices](#)
11. [Layered Architecture](#)
12. [Testing & Production Readiness](#)
13. [Interview Q&A](#)

14. Practice Links & Resources

1. What is Node.js?

Node.js is a **JavaScript runtime** built on Chrome's **V8 engine**. It enables JavaScript to run on the server, not just in browsers.

Characteristic	Description
Runtime	V8 JavaScript engine
I/O Model	Event-driven, non-blocking
Threading	Single-threaded event loop + libuv thread pool
Module system	CommonJS (legacy) + ESM (modern)
Package manager	npm / yarn / pnpm

Why Node.js for Backend?

Advantage	Explanation
Unified language	Same JS/TS on frontend and backend
Non-blocking I/O	Handles thousands of concurrent connections
Huge ecosystem	npm – largest package registry
JSON native	Perfect for REST APIs
Real-time	WebSockets, SSE built-in support

When NOT to Use Node.js

Scenario	Better Alternative
CPU-intensive (video encoding, ML)	Python, Go, Rust + Worker Threads
Heavy computation	Go, Java
Complex transactional systems	Java, C#
Legacy enterprise integration	Java, .NET

2. Event Loop Deep Dive

The event loop is Node.js's core mechanism for handling **concurrency without multi-threading**.

Event Loop Phases

1. timers	← setTimeout, setInterval callbacks
2. pending callbacks	← I/O callbacks deferred to next loop
3. idle, prepare	← internal use only
4. poll	← retrieve new I/O events; block here
5. check	← setImmediate callbacks
6. close callbacks	← socket.on('close', ...)

Between EVERY phase: process microtask queue
(Promise.then, queueMicrotask, process.nextTick*)
* process.nextTick runs BEFORE other microtasks

Macro vs Micro Tasks

Type	Examples	Priority
<code>process.nextTick</code>	<code>process.nextTick(fn)</code>	Highest (runs before microtasks)
Microtasks	<code>Promise.then</code> , <code>queueMicrotask</code>	High (between loop phases)
Macrotasks	<code>setTimeout</code> , <code>setImmediate</code> , I/O	Normal (loop phases)

Code Example: Execution Order

```
console.log("1: sync start");

setTimeout(() => console.log("2: setTimeout"), 0);

Promise.resolve().then(() => console.log("3: promise"));

process.nextTick(() => console.log("4: nextTick"));

setImmediate(() => console.log("5: setImmediate"));

console.log("6: sync end");

// Output:
// 1: sync start
// 6: sync end
// 4: nextTick
// 3: promise
// 2: setTimeout
// 5: setImmediate
```

Non-Blocking I/O Explained

```
const fs = require("fs");

console.log("Start");

// Non-blocking – callback queued, loop continues
fs.readFile("./large-file.txt", "utf8", (err, data) => {
  console.log("File read complete:", data.length);
});

console.log("End – this prints BEFORE file read");

// Output: Start → End → File read complete
```

libuv Thread Pool

Not all operations are truly non-blocking. These use a **4-thread pool** (configurable via `UV_THREADPOOL_SIZE`):

Operation	Thread Pool?
File system (<code>fs.readFile</code>)	✓ Yes
DNS lookup (<code>dns.lookup</code>)	✓ Yes
Crypto (<code>crypto.pbkdf2</code>)	✓ Yes
Network I/O (<code>http.request</code>)	✗ No (OS async)
Database queries	✗ No (network)

Blocking the Event Loop – Anti-Pattern

```
// ❌ BLOCKS the entire event loop – all requests freeze
app.get("/bad", (req, res) => {
  let result = 0;
  for (let i = 0; i < 10_000_000_000; i++) {
    result += i;
  }
  res.json({ result });
});

// ✅ Offload CPU work to Worker Threads
import { Worker } from "worker_threads";

app.get("/good", (req, res) => {
  const worker = new Worker("./heavy-compute.js");
  worker.on("message", result => res.json({ result }));
  worker.on("error", err => res.status(500).json({ error: err.message }));
});
```

Worker Threads Example

```
// heavy-compute.js
import { parentPort, workerData } from "worker_threads";

function fibonacci(n) {
  if (n <= 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}

const result = fibonacci(workerData.n);
parentPort.postMessage(result);
```

3. Modules: CommonJS vs ESM

Comparison

Feature	CommonJS	ESM
Syntax	<code>require()</code> / <code>module.exports</code>	<code>import</code> / <code>export</code>
Loading	Synchronous	Asynchronous
Tree shaking	✗	✓
Top-level await	✗	✓
<code>"type": "module"</code> in <code>package.json</code>	Not needed	Required for .js ESM

ESM (Modern – Use This)

```
// user.service.ts
import { prisma } from "../lib/prisma.js"; // .js extension required in ESM
import type { User } from "../types/user.js";

export async function findUserById(id: string): Promise<User | null> {
  return prisma.user.findUnique({ where: { id } });
}

export default { findUserById };
```

```
// user.controller.ts
import { findUserById } from "./user.service.js";

export async function getUser(req: Request, res: Response) {
  const user = await findUserById(req.params.id);
  if (!user) return res.status(404).json({ error: "User not found" });
  res.json(user);
}
```

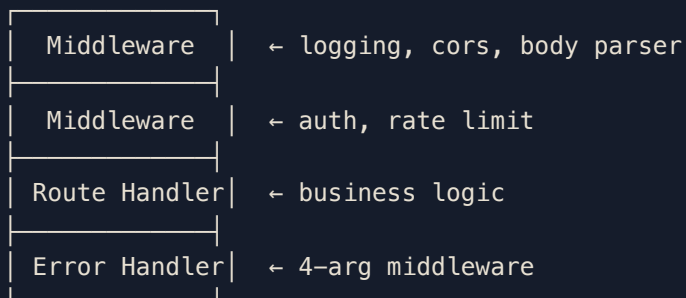
package.json for ESM + TypeScript

```
{
  "name": "my-api",
  "type": "module",
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js"
  }
}
```

4. Express Architecture

Express is a **minimal, unopinionated** web framework for Node.js.

HTTP Request



HTTP Response

Production Server Setup

```
// src/index.ts
import express from "express";
import helmet from "helmet";
import cors from "cors";
import compression from "compression";
import morgan from "morgan";
import { userRouter } from "../routes/user.routes.js";
import { errorHandler } from "../middleware/errorHandler.js";
import { notFoundHandler } from "../middleware/notFoundHandler.js";

const app = express();
const PORT = process.env.PORT ?? 3000;

// Security & performance middleware
app.use(helmet());
app.use(cors({ origin: process.env.ALLOWED_ORIGINS?.split(",") ?? "*" }));
app.use(compression());
app.use(morgan(process.env.NODE_ENV === "production" ? "combined" : "dev"));

// Body parsing
app.use(express.json({ limit: "10mb" }));
app.use(express.urlencoded({ extended: true }));

// Health check (before auth)
app.get("/health", (_req, res) => {
  res.json({ status: "ok", timestamp: new Date().toISOString() });
});

// Routes
app.use("/api/v1/users", userRouter);

// 404 + error handlers (must be last)
app.use(notFoundHandler);
app.use(errorHandler);

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});

export default app;
```

Router Modularization

```
// src/routes/user.routes.ts
import { Router } from "express";
import { validate } from "../middleware/validate.js";
import { authenticate } from "../middleware/authenticate.js";
import { CreateUserSchema, UpdateUserSchema } from "../schemas/user.schema.js";
import * as userController from "../controllers/user.controller.js";

export const userRouter = Router();

userRouter.get("/", userController.listUsers);
userRouter.get("/:id", userController.getUser);
userRouter.post("/", validate(CreateUserSchema), userController.createUser);
userRouter.patch("/:id", authenticate, validate(UpdateUserSchema), userController.updateUser);
userRouter.delete("/:id", authenticate, userController.deleteUser);
```

5. Middleware Patterns

Middleware functions have signature: `(req, res, next) => void` or async variant.

Custom Logger Middleware

```
import { Request, Response, NextFunction } from "express";

export function requestLogger(req: Request, res: Response, next: NextFunction) {
  const start = Date.now();

  res.on("finish", () => {
    const duration = Date.now() - start;
    console.log(`${req.method} ${req.path} ${res.statusCode} ${duration}ms`);
  });

  next();
}
```

Authentication Middleware

```
import jwt from "jsonwebtoken";
import { Request, Response, NextFunction } from "express";

interface AuthRequest extends Request {
  user?: { id: string; role: string };
}

export function authenticate(req: AuthRequest, res: Response, next: NextFunction) {
  const authHeader = req.headers.authorization;

  if (!authHeader?.startsWith("Bearer ")) {
    return res.status(401).json({ error: "Missing or invalid token" });
  }

  const token = authHeader.slice(7);

  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET!) as { sub: string; role: string };
    req.user = { id: payload.sub, role: payload.role };
    next();
  } catch {
    return res.status(401).json({ error: "Token expired or invalid" });
  }
}

export function authorize(...roles: string[]) {
  return (req: AuthRequest, res: Response, next: NextFunction) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return res.status(403).json({ error: "Insufficient permissions" });
    }
    next();
  };
}
```

Rate Limiting

```
import rateLimit from "express-rate-limit";

export const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100,
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: "Too many requests, please try again later." },
});

export const authLimiter = rateLimit({
  windowMs: 60 * 60 * 1000, // 1 hour
  max: 5,
  message: { error: "Too many login attempts." },
});

// Usage
app.use("/api/", apiLimiter);
app.use("/api/auth/login", authLimiter);
```

Middleware Execution Order

Order	Middleware	Why
1	helmet , cors	Security headers first
2	compression	Before response sent
3	morgan / logger	Log all requests
4	Body parsers	Before route handlers read body
5	Rate limiter	Before expensive handlers
6	Auth middleware	Per-route or route group
7	Route handlers	Business logic
8	404 handler	Catch unknown routes
9	Error handler	Must have 4 parameters

6. REST API Design

REST Principles

Principle	Implementation
Resource-based URLs	<code>/users</code> , <code>/users/123/orders</code>
HTTP verbs	GET (read), POST (create), PUT/PATCH (update), DELETE
Stateless	No server-side session; use JWT
Representations	JSON (primary), XML (legacy)
Status codes	Semantic HTTP codes

URL Design Rules

✓ Good

GET `/api/v1/users`

GET `/api/v1/users/123`

POST `/api/v1/users`

GET `/api/v1/users/123/orders`

✗ Bad

GET `/api/getUsers`

GET `/api/user?id=123`

POST `/api/v1/createUser`

GET `/api/v1/orders?userId=123`

HTTP Status Codes Reference

Code	Meaning	When to Use
200	OK	Successful GET, PATCH
201	Created	Successful POST
204	No Content	Successful DELETE
400	Bad Request	Validation failure
401	Unauthorized	Missing/invalid auth
403	Forbidden	Valid auth, insufficient permissions
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate email, version conflict
422	Unprocessable Entity	Semantic validation error
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unhandled exception

Pagination, Filtering, Sorting

```
// GET /api/v1/users?page=2&limit=20&role=admin&sort=-createdAt&search=john

export async function listUsers(req: Request, res: Response) {
  const page = Math.max(1, Number(req.query.page) || 1);
  const limit = Math.min(100, Number(req.query.limit) || 20);
  const skip = (page - 1) * limit;

  const where = {
    ...(req.query.role && { role: req.query.role as string }),
    ...(req.query.search && {
      OR: [
        { name: { contains: req.query.search as string, mode: "insensitive" as const },
        { email: { contains: req.query.search as string, mode: "insensitive" as const }
      ],
    })),
  };

  const sortField = (req.query.sort as string)?.replace(/^-/, "") ?? "createdAt";
  const sortOrder = (req.query.sort as string)?.startsWith("-") ? "desc" : "asc";

  const [users, total] = await Promise.all([
    prisma.user.findMany({ where, skip, take: limit, orderBy: { [sortField]: sortOrder },
    prisma.user.count({ where })),
  ]);

  res.json({
    data: users,
    meta: {
      page,
      limit,
      total,
      totalPages: Math.ceil(total / limit),
      hasNext: page * limit < total,
      hasPrev: page > 1,
    },
  });
}
```

API Versioning Strategies

Strategy	Example	Pros	Cons
URL path	<code>/api/v1/users</code>	Simple, explicit	URL changes
Header	<code>Accept: application/vnd.api.v2+json</code>	Clean URLs	Harder to test
Query param	<code>/api/users?version=2</code>	Easy	Messy

Consistent Response Envelope

```
// Success
{ "data": { ... }, "meta": { "page": 1, "total": 100 } }

// Error
{ "error": { "code": "VALIDATION_ERROR", "message": "...", "details": [...] } }
```

7. Validation & Sanitization

Never trust client input. Validate at the **boundary** (controller/route handler).

Zod Validation Middleware

```
// src/schemas/user.schema.ts
import { z } from "zod";

export const CreateUserSchema = z.object({
  name: z.string().min(1).max(100).trim(),
  email: z.string().email().toLowerCase(),
  password: z.string().min(8).max(128),
  role: z.enum(["user", "admin"]).default("user"),
});

export const UpdateUserSchema = CreateUserSchema.partial().omit({ password: true });

export type CreateUserInput = z.infer<typeof CreateUserSchema>;
export type UpdateUserInput = z.infer<typeof UpdateUserSchema>;
```

```
// src/middleware/validate.ts
import { Request, Response, NextFunction } from "express";
import { ZodSchema } from "zod";

export function validate(schema: ZodSchema) {
  return (req: Request, res: Response, next: NextFunction) => {
    const result = schema.safeParse(req.body);

    if (!result.success) {
      return res.status(400).json({
        error: {
          code: "VALIDATION_ERROR",
          message: "Invalid request body",
          details: result.error.flatten().fieldErrors,
        },
      });
    }

    req.body = result.data; // typed, sanitized body
    next();
  };
}
```

Input Sanitization Checklist

Field	Sanitization
Email	Lowercase, trim
Strings	Trim whitespace, max length
HTML content	Escape or use DOMPurify
IDs	Validate UUID/ObjectId format
Numbers	Parse and range-check
Arrays	Max length, validate each item

8. Error Handling

Custom Error Classes

```
// src/errors/AppError.ts
export class AppError extends Error {
  constructor(
    public statusCode: number,
    public code: string,
    message: string,
    public isOperational = true
  ) {
    super(message);
    this.name = this.constructor.name;
    Error.captureStackTrace(this, this.constructor);
  }
}

export class NotFoundError extends AppError {
  constructor(resource: string) {
    super(404, "NOT_FOUND", `${resource} not found`);
  }
}

export class ValidationError extends AppError {
  constructor(message: string, public details?: unknown) {
    super(400, "VALIDATION_ERROR", message);
  }
}

export class UnauthorizedError extends AppError {
  constructor(message = "Unauthorized") {
    super(401, "UNAUTHORIZED", message);
  }
}
```

Global Error Handler

```
// src/middleware/errorHandler.ts
import { Request, Response, NextFunction } from "express";
import { AppError } from "../errors/AppError.js";
import { ZodError } from "zod";

export function errorHandler(
  err: Error,
  req: Request,
  res: Response,
  _next: NextFunction
) {
  // Log error (use structured logging in production)
  console.error({
    message: err.message,
    stack: err.stack,
    path: req.path,
    method: req.method,
  });

  if (err instanceof AppError) {
    return res.status(err.statusCode).json({
      error: { code: err.code, message: err.message },
    });
  }

  if (err instanceof ZodError) {
    return res.status(400).json({
      error: {
        code: "VALIDATION_ERROR",
        message: "Invalid input",
        details: err.flatten().fieldErrors,
      },
    });
  }

  // Prisma errors
  if (err.name === "PrismaClientKnownRequestError") {
    const prismaErr = err as { code: string };
    if (prismaErr.code === "P2002") {
      return res.status(409).json({ error: { code: "DUPLICATE", message: "Record a" } });
    }
  }

  // Unknown errors – don't leak internals
  res.status(500).json({
    error: { code: "INTERNAL_ERROR", message: "Something went wrong" },
  });
}
```

```
});  
}
```

Async Error Wrapper

```
// Avoid try/catch in every controller  
import { Request, Response, NextFunction, RequestHandler } from "express";  
  
export function asyncHandler(fn: RequestHandler): RequestHandler {  
  return (req, res, next) => {  
    Promise.resolve(fn(req, res, next)).catch(next);  
  };  
}  
  
// Usage  
export const getUser = asyncHandler(async (req, res) => {  
  const user = await userService.findById(req.params.id);  
  if (!user) throw new NotFoundError("User");  
  res.json({ data: user });  
});
```

Error Handling Rules

Rule	Reason
Operational errors → 4xx	Client can fix (validation, auth)
Programming errors → 5xx	Server bug; log and alert
Never expose stack traces in production	Security risk
Always call <code>next(err)</code> in middleware	Propagates to error handler
Error handler must have 4 params	Express identifies it by arity

9. Streams & Buffers

Why Streams?

Approach	Memory for 1GB file
Read entire file	~1GB RAM
Stream	~64KB RAM (chunk size)

Stream Types

Type	Direction	Example
Readable	Source	<code>fs.createReadStream</code> , HTTP request body
Writable	Destination	<code>fs.createWriteStream</code> , HTTP response
Duplex	Both	TCP socket, WebSocket
Transform	Modify while piping	gzip, crypto cipher

File Upload with Streams

```
import fs from "fs";
import path from "path";
import { pipeline } from "stream/promises";
import { createWriteStream } from "fs";

app.post("/upload", async (req, res, next) => {
  const filename = `upload-${Date.now()}.csv`;
  const dest = path.join("./uploads", filename);

  try {
    await pipeline(req, createWriteStream(dest));
    res.json({ message: "Upload complete", filename });
  } catch (err) {
    next(err);
  }
});
```

Transform Stream Example

```
import { Transform } from "stream";

const upperCaseTransform = new Transform({
  transform(chunk, _encoding, callback) {
    callback(null, chunk.toString().toUpperCase());
  },
});

fs.createReadStream("input.txt")
  .pipe(upperCaseTransform)
  .pipe(fs.createWriteStream("output.txt"));
```

Buffers

```
// Buffer – raw binary data
const buf = Buffer.from("Hello", "utf8");
console.log(buf.toString("hex")); // 48656c6c6f
console.log(buf.length);          // 5 bytes

// Concatenate buffers
const combined = Buffer.concat([buf, Buffer.from(" World")]);
```

10. Security Best Practices

OWASP Top 10 Mitigations in Express

Vulnerability	Mitigation
Injection (SQL, NoSQL)	Parameterized queries, ORM, input validation
Broken Authentication	JWT httpOnly cookies, bcrypt, rate limiting
Sensitive Data Exposure	HTTPS, env vars, no secrets in code
XSS	Escape output, CSP headers, httpOnly cookies
CSRF	SameSite cookies, CSRF tokens for forms
Security Misconfiguration	Helmet, disable X-Powered-By
Broken Access Control	RBAC middleware, check ownership
SSRF	Validate URLs, allowlist domains

Security Middleware Stack

```
import helmet from "helmet";
import mongoSanitize from "express-mongo-sanitize";
import hpp from "hpp";

app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ['self'],
      scriptSrc: ['self'],
      styleSrc: ['self', 'unsafe-inline'],
    },
  },
}));

app.use(mongoSanitize()); // prevent NoSQL injection
app.use(hpp());           // prevent HTTP parameter pollution

// Disable X-Powered-By
app.disable("x-powered-by");
```

Password Hashing

```
import bcrypt from "bcrypt";

const SALT_ROUNDS = 12;

export async function hashPassword(password: string): Promise<string> {
  return bcrypt.hash(password, SALT_ROUNDS);
}

export async function verifyPassword(password: string, hash: string): Promise<bool> {
  return bcrypt.compare(password, hash);
}
```

Environment Variables

```
// src/config/env.ts – validate env at startup
import { z } from "zod";

const EnvSchema = z.object({
  NODE_ENV: z.enum(["development", "production", "test"]).default("development"),
  PORT: z.coerce.number().default(3000),
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
  JWT_EXPIRES_IN: z.string().default("7d"),
});

export const env = EnvSchema.parse(process.env);
// App crashes immediately if env is invalid – fail fast
```

11. Layered Architecture

Production Express apps use **separation of concerns** across layers.

Routes Layer	HTTP routing, middleware chain
Controllers Layer	Request/response handling
Services Layer	Business logic
Repositories Layer	Data access (DB queries)
Database / External	PostgreSQL, MongoDB, Redis

Layer Responsibilities

Layer	Responsibility	Should NOT
Routes	URL mapping, middleware	Contain business logic
Controllers	Parse request, call service, send response	Direct DB queries
Services	Business rules, orchestration	Know about HTTP
Repositories	CRUD, queries	Contain business logic

Full Example: User Module

```
// src/repositories/user.repository.ts
import { prisma } from "../../lib/prisma.js";
import type { CreateUserInput } from "../../schemas/user.schema.js";

export const userRepository = {
  findById: (id: string) => prisma.user.findUnique({ where: { id } }),
  findByEmail: (email: string) => prisma.user.findUnique({ where: { email } }),
  findMany: (skip: number, take: number) =>
    prisma.user.findMany({ skip, take, orderBy: { createdAt: "desc" } }),
  count: () => prisma.user.count(),
  create: (data: CreateUserInput & { passwordHash: string }) =>
    prisma.user.create({ data, select: { id: true, name: true, email: true, role: true } }),
  delete: (id: string) => prisma.user.delete({ where: { id } }),
};
```

```
// src/services/user.service.ts
import { userRepository } from "../repositories/user.repository.js";
import { hashPassword } from "../utils/password.js";
import { NotFoundError, ValidationError } from "../errors/AppError.js";
import type { CreateUserInput } from "../schemas/user.schema.js";

export const userService = {
  async getById(id: string) {
    const user = await userRepository.findById(id);
    if (!user) throw new NotFoundError("User");
    return user;
  },

  async list(page: number, limit: number) {
    const skip = (page - 1) * limit;
    const [users, total] = await Promise.all([
      userRepository.findMany(skip, limit),
      userRepository.count(),
    ]);
    return { users, total, page, limit };
  },

  async create(input: CreateUserInput) {
    const existing = await userRepository.findByEmail(input.email);
    if (existing) throw new ValidationError("Email already registered");

    const passwordHash = await hashPassword(input.password);
    return userRepository.create({ ...input, passwordHash });
  },

  async delete(id: string) {
    await this.getById(id); // throws if not found
    return userRepository.delete(id);
  },
};
```

```
// src/controllers/user.controller.ts
import { Request, Response } from "express";
import { userService } from "../services/user.service.js";
import { asyncHandler } from "../utils/asyncHandler.js";

export const getUser = asyncHandler(async (req: Request, res: Response) => {
  const user = await userService.getById(req.params.id);
  res.json({ data: user });
});

export const listUsers = asyncHandler(async (req: Request, res: Response) => {
  const page = Number(req.query.page) || 1;
  const limit = Number(req.query.limit) || 20;
  const result = await userService.list(page, limit);
  res.json({ data: result.users, meta: { page: result.page, limit: result.limit, total: result.total } });
});

export const createUser = asyncHandler(async (req: Request, res: Response) => {
  const user = await userService.create(req.body);
  res.status(201).json({ data: user });
});

export const deleteUser = asyncHandler(async (req: Request, res: Response) => {
  await userService.delete(req.params.id);
  res.status(204).send();
});
```

Project Structure

```
src/  
├── index.ts  
├── config/  
│   └── env.ts  
├── routes/  
│   ├── index.ts  
│   └── user.routes.ts  
├── controllers/  
│   └── user.controller.ts  
├── services/  
│   └── user.service.ts  
├── repositories/  
│   └── user.repository.ts  
├── middleware/  
│   ├── authenticate.ts  
│   ├── validate.ts  
│   └── errorHandler.ts  
├── schemas/  
│   └── user.schema.ts  
├── errors/  
│   └── AppError.ts  
└── utils/  
    ├── asyncHandler.ts  
    └── password.ts
```

12. Testing & Production Readiness

Testing Pyramid

Level	Tool	What to Test
Unit	Vitest/Jest	Services, utils, validators
Integration	Supertest	Routes + DB
E2E	Playwright	Full user flows

Integration Test Example

```
import request from "supertest";
import app from "../src/index.js";
import { prisma } from "../src/lib/prisma.js";

describe("POST /api/v1/users", () => {
  afterEach(async () => {
    await prisma.user.deleteMany();
  });

  it("creates a user with valid input", async () => {
    const res = await request(app)
      .post("/api/v1/users")
      .send({ name: "Amit", email: "amit@test.com", password: "password123" })
      .expect(201);

    expect(res.body.data.email).toBe("amit@test.com");
    expect(res.body.data.password).toBeUndefined();
  });

  it("returns 400 for invalid email", async () => {
    const res = await request(app)
      .post("/api/v1/users")
      .send({ name: "Amit", email: "not-an-email", password: "password123" })
      .expect(400);

    expect(res.body.error.code).toBe("VALIDATION_ERROR");
  });
});
```

Graceful Shutdown

```
const server = app.listen(PORT);

process.on("SIGTERM", async () => {
  console.log("SIGTERM received, shutting down gracefully");
  server.close(async () => {
    await prisma.$disconnect();
    process.exit(0);
  });
});
```

13. Interview Q&A

Q1: Explain the Node.js event loop.

Answer: Node.js uses a single-threaded event loop to handle async I/O. When an async operation (file read, DB query) starts, Node delegates it to libuv/OS and continues executing. When complete, the callback enters the appropriate queue (timers, poll, check). Microtasks (Promises) run between phases. This allows handling thousands of concurrent connections without thread-per-request overhead.

Q2: What is the difference between `setImmediate` and `setTimeout(fn, 0)` ?

Answer: `setTimeout(fn, 0)` goes to the timers phase (minimum 1ms delay in Node). `setImmediate` runs in the check phase, after the poll phase completes. Inside an I/O callback, `setImmediate` always runs before `setTimeout`. At the top level, order is non-deterministic.

Q3: How does Express middleware work?

Answer: Middleware functions form a pipeline. Each receives `(req, res, next)`. Calling `next()` passes control to the next middleware. If middleware doesn't call `next()` or send a response, the request hangs. Error-handling middleware has 4 parameters `(err, req, res, next)` and must be registered last.

Q4: How do you prevent common security vulnerabilities?

Answer: Use Helmet for security headers, validate/sanitize all input with Zod, parameterize DB queries via ORM, hash passwords with bcrypt, store JWTs in httpOnly cookies, rate-limit auth endpoints, keep dependencies updated, validate env vars at startup, and never expose stack traces in production.

Q5: What is the difference between PUT and PATCH?

Answer: PUT replaces the entire resource — client sends full representation. PATCH applies partial updates — only changed fields. Use PATCH for most update operations; PUT when

replacing the whole resource.

Q6: How do you handle errors in async Express handlers?

Answer: Wrap async handlers with an `asyncHandler` utility that catches rejected Promises and calls `next(err)`. Alternatively use `express-async-errors` package. Without this, unhandled rejections crash the process or leave requests hanging.

Q7: Explain layered architecture benefits.

Answer: Separates HTTP concerns (controllers), business logic (services), and data access (repositories). Benefits: testability (mock repositories in service tests), maintainability (change DB without touching controllers), reusability (services callable from CLI, jobs, other services).

Q8: When would you use streams?

Answer: When processing large files or data that doesn't fit in memory — file uploads/downloads, video processing, log parsing, CSV import/export. Streams process data in chunks (~64KB), keeping memory constant regardless of file size.

14. Practice Links & Resources

Official Documentation

- [Node.js Docs](#) — API reference
- [Express.js Guide](#) — Routing, middleware
- [Node.js Design Patterns](#) — Book by Mammino & Luciano

Interactive Learning

- [Node.js Event Loop Visualizer](#) — Philip Roberts talk
- [Express Middleware Practice](#) — Official examples

Practice Projects

Project	Skills
REST API with auth	JWT, bcrypt, middleware, layered arch
File upload service	Streams, multer, S3
Real-time chat	WebSockets, Socket.io, rooms
URL shortener	Redis caching, rate limiting
E-commerce API	CRUD, pagination, transactions

npm Packages to Know

Package	Purpose
<code>express</code>	Web framework
<code>helmet</code>	Security headers
<code>cors</code>	Cross-origin requests
<code>zod</code>	Runtime validation
<code>bcrypt</code>	Password hashing
<code>jsonwebtoken</code>	JWT auth
<code>express-rate-limit</code>	Rate limiting
<code>morgan</code>	HTTP logging
<code>supertest</code>	HTTP testing
<code>winston / pino</code>	Structured logging

Next file: [06-databases-complete.md](#)

Product Company Interview Bank – Node.js and Express (100 Questions)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Target companies: Google, Meta, Microsoft, Amazon, Adobe, Atlassian, Uber, Flipkart **Rule:** Full terms only — spell out JWT as JSON Web Token, API as Application Programming Interface, etc.

Question 1: What is Node.js?

Answer: Node.js is a JavaScript runtime built on Chrome's V8 engine that lets you run JavaScript outside the browser — on servers, command-line tools, and devices. It uses an event-driven, non-blocking Input/Output model, making it efficient for handling many concurrent connections. Companies use Node.js for Application Programming Interface (API) servers, real-time apps, and microservices because one language (JavaScript) spans frontend and backend.

Question 2: What is the event loop in Node.js?

Answer: The event loop is how Node.js handles asynchronous operations on a single thread. When you call async functions (file read, database query, network request), Node delegates work to the system or thread pool and continues executing other code. When work completes, callbacks enter queues and the event loop processes them in phases: timers, pending callbacks, poll, check, and close callbacks. This lets one thread serve thousands of connections efficiently.

Question 3: What are microtasks vs macrotasks in Node.js?

Answer: Microtasks (Promise callbacks, `queueMicrotask`) run after the current operation completes and before the next event loop phase — they have higher priority. Macrotasks (`setTimeout`, `setImmediate`, Input/Output callbacks) run in event loop phases.

`process.nextTick` runs even before microtasks — use sparingly as it can starve the event loop. Order matters: synchronous code → `process.nextTick` → microtasks → macrotasks.

Question 4: Is Node.js single-threaded?

Answer: JavaScript execution in Node.js is single-threaded — one call stack runs your code. However, Node.js uses libuv's thread pool (default 4 threads) for blocking operations like file system access, DNS lookup, and some cryptography. Network Input/Output is handled asynchronously by the operating system. For CPU-heavy work, use worker threads or separate processes to avoid blocking the event loop.

Question 5: What happens if you block the event loop?

Answer: Blocking the event loop freezes the entire server — no requests get processed, timers stall, and clients timeout. Common blockers: synchronous file reads (`readFileSync`), heavy computation in a loop, large JSON parsing, or complex regex on big strings. Fix: use async APIs, offload CPU work to worker threads, or break work into chunks. Always profile with `--inspect` or logging if latency spikes.

Question 6: What are Worker Threads in Node.js?

Answer: Worker Threads (`worker_threads` module) run JavaScript in parallel threads for CPU-intensive tasks — image processing, encryption, data crunching — without blocking the main event loop. Each worker has its own V8 instance. Communication happens via message passing. Use when work is CPU-bound; stick to async Input/Output for network and file operations on the main thread.

Question 7: What is the difference between Node.js and Express?

Answer: Node.js is the runtime — it executes JavaScript and provides built-in modules (`http` , `fs` , `path`). Express is a minimal web framework built on Node.js that simplifies building web

servers and Application Programming Interfaces. Express adds routing, middleware, request parsing, and response helpers. You can build APIs with raw Node.js `http`, but Express (or Fastify, Koa) saves boilerplate and is industry standard.

Question 8: What is middleware in Express?

Answer: Middleware are functions that run between receiving a request and sending a response. Each middleware receives `(req, res, next)`. It can read/modify the request, end the response, or call `next()` to pass control to the next middleware. Examples: logging, parsing JSON body, authentication checks, error handling. Order matters — middleware runs in the order it is registered.

Question 9: What are common built-in and third-party Express middleware?

Answer: Built-in (Express 4.16+): `express.json()` parses JSON bodies, `express.urlencoded()` parses form data, `express.static()` serves files. Popular third-party: `cors` for Cross-Origin Resource Sharing, `helmet` for security headers, `morgan` for HTTP logging, `compression` for gzip, `cookie-parser` for cookies. Apply global middleware early in `app.use()` before routes.

Question 10: How do you create a basic Express server?

Answer: Install Express, import it, create an app, define routes, and listen on a port:

```
const express = require('express');
const app = express();
app.use(express.json());

app.get('/health', (req, res) => {
  res.json({ status: 'ok' });
});

app.listen(3000, () => console.log('Server on port 3000'));
```

This is the foundation for every Express Application Programming Interface.

Question 11: What is REST (Representational State Transfer)?

Answer: REST is an architectural style for designing networked Application Programming Interfaces. Resources (users, orders) are identified by URLs. Standard HTTP methods map to actions: GET (read), POST (create), PUT/PATCH (update), DELETE (remove). Responses use HTTP status codes (200 success, 201 created, 404 not found, 500 server error). RESTful APIs are stateless — each request contains all information needed.

Question 12: What are HTTP status codes you should know?

Answer: **2xx Success:** 200 OK, 201 Created, 204 No Content. **3xx Redirect:** 301 Moved Permanently, 304 Not Modified. **4xx Client Error:** 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity. **5xx Server Error:** 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable. Use precise codes — 401 when not authenticated, 403 when authenticated but not allowed.

Question 13: How do you structure a REST API in Express?

Answer: Organize by resource with routers: `routes/users.js`, `routes/posts.js`. Mount with `app.use('/api/users', userRouter)`. Use appropriate HTTP methods on each route. Separate concerns: routes handle HTTP, controllers contain logic, services talk to databases, models define data shape. Example: `GET /api/users/:id` → `userController.getById` → `userService.findById` → MongoDB query.

Question 14: What is the difference between `app.get` and `router.get` ?

Answer: `app.get('/path', handler)` registers a route directly on the Express application. `router.get('/path', handler)` registers on a Router instance, which you mount with `app.use('/prefix', router)`. Routers modularize routes — a `userRouter` handles all

`/users` endpoints in one file. Routers can have their own middleware applied to all routes in that module.

Question 15: How do you handle route parameters and query strings?

Answer: Route parameters: `app.get('/users/:id', (req, res) => { const id = req.params.id })` — part of the URL path. Query strings: `GET /search?q=node&page=2` accessed via `req.query.q` and `req.query.page`. Body data (POST/PUT): parsed by `express.json()` into `req.body`. Always validate and sanitize all three — never trust client input.

Question 16: What is request validation and why is it important?

Answer: Request validation checks that incoming data matches expected shape, types, and rules before processing. Prevents crashes, security holes, and bad database records. Use libraries like Joi, Zod, or express-validator. Example: email must be valid format, password minimum 8 characters, age must be a number. Return 400 Bad Request with clear error messages when validation fails.

Question 17: How do you handle errors in Express?

Answer: Use try/catch in async route handlers and pass errors to Express via `next(error)`. Define an error-handling middleware with four arguments: `(err, req, res, next) => { res.status(err.status || 500).json({ message: err.message }) }`. Register it last, after all routes. For async functions, wrap handlers or use `express-async-errors` to auto-forward rejections. Never expose stack traces in production.

Question 18: What is CORS (Cross-Origin Resource Sharing)?

Answer: CORS is a browser security mechanism that blocks web pages from calling APIs on a different domain unless the server explicitly allows it. Your React app on `localhost:3000`

calling an API on `localhost:5000` needs CORS headers. Use the `cors` package:
`app.use(cors({ origin: 'https://myapp.com' })))`. Server-to-server calls are not restricted by CORS — only browsers enforce it.

Question 19: What is the difference between CommonJS and ES Modules in Node.js?

Answer: CommonJS uses `require()` and `module.exports` — the traditional Node.js system. ES Modules (ESM) use `import` and `export` — the JavaScript standard. Enable ESM with `"type": "module"` in `package.json` or `.mjs` file extension. Modern projects prefer ESM for consistency with frontend code. Node.js supports both; mixing requires care (`createRequire` for importing CommonJS in ESM).

Question 20: What is `npm` and what is `package.json` ?

Answer: npm (Node Package Manager) is the default registry and tool for installing JavaScript packages. `package.json` lists project metadata, dependencies, scripts (`"start": "node index.js"`), and version info. `package-lock.json` locks exact dependency versions for reproducible installs. Use `npm install express` to add packages; `npm run start` to run scripts. Alternatives: yarn, pnpm.

Question 21: What is the Node.js `fs` module?

Answer: The `fs` (file system) module reads, writes, and manipulates files and directories. Always prefer async methods: `fs.promises.readFile('file.txt', 'utf8')` or `fs.readFile('file.txt', callback)`. Sync methods block the event loop. Common operations: `readFile`, `writeFile`, `readdir`, `mkdir`, `unlink` (delete). For streams on large files, use `createReadStream` instead of reading entire file into memory.

Question 22: What are Streams in Node.js?

Answer: Streams process data piece by piece instead of loading everything into memory. Types: Readable (read file/network), Writable (write output), Duplex (both), Transform (modify while passing through). Example: reading a 1 GB file and piping to HTTP response without using 1 GB RAM. Use `pipe()` to connect streams: `readStream.pipe(writeStream)`. Essential for video, logs, and large file uploads.

Question 23: What is the `path` module used for?

Answer: The `path` module handles file and directory paths in a cross-platform way. Key methods: `path.join('users', id, 'avatar.png')` builds paths safely, `path.resolve()` gets absolute paths, `path.extname('file.js')` returns `.js`, `path.basename()` gets filename. Always use `path.join` instead of string concatenation — Windows uses backslashes while Linux/Mac use forward slashes.

Question 24: What is the `http` module vs Express?

Answer: Node's built-in `http` module creates servers with low-level control: `http.createServer((req, res) => { ... })`. You manually parse URLs, read bodies, and set headers. Express wraps `http` with routing, middleware, and helpers like `res.json()`. Use raw `http` for learning or minimal services; Express for real Application Programming Interfaces where developer productivity matters.

Question 25: What is environment-based configuration?

Answer: Store settings like database URLs, API keys, and port numbers in environment variables, not hardcoded in source code. Use `process.env.PORT || 3000` and a `.env` file with `dotenv` package (loaded at startup). Different environments (development, staging, production) get different values. Never commit `.env` to git — use `.env.example` as a template. Platforms like AWS and Heroku inject env vars at deploy time.

Question 26: How do you connect Node.js to MongoDB?

Answer: Use the official MongoDB driver or Mongoose (Object Document Mapper). With Mongoose: install mongoose, connect once at startup with `mongoose.connect(process.env.MONGODB_URI)`, define schemas and models, then query in routes. Connection pooling is handled automatically. Handle connection errors and disconnections. For serverless, consider connection caching to avoid opening too many connections per request.

Question 27: What is Mongoose and what does it provide?

Answer: Mongoose is an Object Document Mapper (ODM) for MongoDB and Node.js. It defines schemas (field types, validation, defaults), models (constructors for documents), middleware (pre/post hooks), and query helpers. Example: a User schema with required email and hashed password. Mongoose adds structure to MongoDB's flexible documents and is standard in MERN stack tutorials and production apps.

Question 28: What is the difference between SQL and MongoDB for Node.js developers?

Answer: SQL databases (PostgreSQL, MySQL) store data in tables with fixed schemas and use Structured Query Language. MongoDB stores JSON-like documents in collections with flexible schemas. Node.js works well with both — use Mongoose/Prisma for MongoDB, Sequelize/Prisma for SQL. MongoDB suits rapid prototyping and nested data; SQL suits complex relationships and transactions. Many companies use PostgreSQL even in "MERN" stacks (replacing M with PostgreSQL).

Question 29: What is bcrypt and why use it for passwords?

Answer: bcrypt is a password hashing library — it converts plain passwords into one-way hashes that cannot be reversed. Never store plain text passwords. bcrypt adds a salt (random data) and is intentionally slow to resist brute-force attacks. In Node.js: `const hash = await`

`bcrypt.hash(password, 12)` to store, `await bcrypt.compare(password, hash)` to verify on login. Use cost factor 10–12 for production.

Question 30: What is JSON Web Token (JWT) authentication in Express?

Answer: JSON Web Token (JWT) is a compact token containing encoded user claims (user ID, role) signed with a secret. Flow: user logs in with credentials → server verifies → server sends JWT → client sends JWT in `Authorization: Bearer <token>` header on later requests → middleware verifies signature and attaches user to `req.user`. JWTs are stateless — no server-side session storage needed, but revocation is harder.

Question 31: How do you write authentication middleware in Express?

Answer: Create middleware that reads the token from headers, verifies it with `jsonwebtoken.verify(token, secret)`, and calls `next()` or returns 401:

```
function authMiddleware(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'No token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ message: 'Invalid token' });
  }
}
```

Apply to protected routes: `router.get('/profile', authMiddleware, getProfile)`.

Question 32: What is the difference between authentication and authorization?

Answer: Authentication verifies who you are — login with username/password or OAuth. Authorization verifies what you are allowed to do — can this user delete this post?

Authentication happens first (401 Unauthorized if failed). Authorization checks roles or permissions after auth (403 Forbidden if not allowed). Example: logged-in user (authenticated) who is not an admin trying to access admin panel (authorization fails).

Question 33: What are sessions vs tokens?

Answer: Sessions store user state on the server (memory, Redis) with a session ID in a cookie — easy to revoke, harder to scale across servers. Tokens (JWT) store claims in the token itself — stateless, scales easily, but harder to invalidate before expiry. Sessions suit traditional web apps; JWTs suit Single Page Applications and mobile apps. Many apps use short-lived JWTs with refresh tokens stored in HTTP-only cookies.

Question 34: What is rate limiting and how do you implement it?

Answer: Rate limiting restricts how many requests a client can make in a time window — prevents abuse, brute-force login, and denial-of-service. Use `express-rate-limit`: `app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 100 })))` allows 100 requests per 15 minutes per Internet Protocol (IP). Apply stricter limits on auth endpoints. In production, use Redis-backed rate limiting for distributed servers.

Question 35: What is `helmet` in Express?

Answer: Helmet is middleware that sets security-related HTTP headers: Content Security Policy, X-Frame-Options (clickjacking protection), Strict-Transport-Security, and others. One line: `app.use(helmet())`. It does not replace proper authentication or input validation but adds defense-in-depth. Standard in production Express apps and often asked in security-focused interviews.

Question 36: How do you handle file uploads in Express?

Answer: Use `multer` middleware for multipart form data. Configure storage (disk or memory), file size limits, and file type filters. Example: `upload.single('avatar')` adds file to `req.file`. Validate file types (only images), limit size (5 MB), scan for malware in production, and store in cloud storage (Amazon S3, Cloudinary) rather than local disk for scalability. Never execute uploaded files.

Question 37: What is logging in Node.js applications?

Answer: Logging records events, errors, and request info for debugging and monitoring. Avoid `console.log` in production — use structured loggers like Winston or Pino that output JSON with timestamps, levels (info, warn, error), and request IDs. HTTP logging middleware (`morgan` or Pino-http) logs each request. Send logs to centralized systems (Datadog, CloudWatch, ELK stack) in production.

Question 38: What is clustering in Node.js?

Answer: The `cluster` module forks worker processes sharing the same port — each runs on a separate CPU core. The primary process distributes incoming connections to workers. This uses multi-core machines since a single Node.js process uses one core. Example: `cluster.fork()` for each CPU. Alternatives: PM2 process manager with cluster mode, or Kubernetes scaling multiple containers.

Question 39: What is PM2?

Answer: PM2 is a production process manager for Node.js. It keeps apps running (auto-restart on crash), supports cluster mode for multi-core, manages logs, and provides zero-downtime reloads. Commands: `pm2 start app.js`, `pm2 list`, `pm2 logs`. Use with ecosystem config files for environment variables and deploy scripts. Common in VPS deployments before container orchestration.

Question 40: What is the difference between `process.env` and command-line arguments?

Answer: `process.env` holds environment variables set by the operating system, shell, or hosting platform — ideal for secrets and config that changes per environment. Command-line arguments (`process.argv`) are values passed when starting the script — useful for one-off scripts (`node migrate.js --up`). For web servers, prefer environment variables; never pass secrets as CLI args (visible in process lists).

Question 41: What are Promises and `async/await` in Node.js?

Answer: Promises represent future completion of async work — `.then()` for success, `.catch()` for errors. `async/await` is syntactic sugar over Promises — `async function` always returns a Promise; `await` pauses until resolved. Use `async/await` for readable sequential code. Always wrap in `try/catch` or use `.catch()` to handle rejections — unhandled rejections can crash Node.js.

Question 42: What is the difference between `setImmediate` and `setTimeout(fn, 0)` ?

Answer: Both defer execution but run in different event loop phases. `setTimeout(fn, 0)` runs in the timers phase (minimum 1 ms delay in Node.js). `setImmediate(fn)` runs in the check phase, after poll — typically before `setTimeout(0)` when called from the main module. Inside Input/Output callbacks, order can vary. For deferring work after current operations, prefer `setImmediate` in Node.js-specific code.

Question 43: What is an Application Programming Interface (API) version strategy?

Answer: Version your API when making breaking changes so existing clients keep working. Common approaches: URL path (`/api/v1/users`), header (`Accept: application/vnd.myapp.v1+json`), or query param. Start with v1, increment on breaking

changes (removed fields, changed behavior). Deprecate old versions with sunset headers. Document changes in a changelog. Companies like Stripe maintain multiple API versions for years.

Question 44: What is pagination in REST APIs?

Answer: Pagination splits large result sets into pages instead of returning thousands of records at once. Offset pagination: `GET /users?page=2&limit=20` — simple but slow on large offsets. Cursor pagination: `GET /users?cursor=abc123&limit=20` — faster for infinite scroll, uses a bookmark ID. Return metadata: total count, next cursor, hasMore. Default limits (e.g., max 100) prevent abuse.

Question 45: What is idempotency in HTTP methods?

Answer: An idempotent operation produces the same result no matter how many times you call it. GET, PUT, DELETE are idempotent — calling DELETE twice on the same resource still leaves it deleted. POST is not idempotent — two identical POST requests create two resources. Important for retries: if a network request fails, safely retry idempotent methods; for POST payments, use idempotency keys to prevent duplicate charges.

Question 46: How do you test Express APIs?

Answer: Use Jest or Mocha with Supertest — it sends HTTP requests to your Express app without starting a real server: `request(app).get('/api/users').expect(200)`. Mock database calls with libraries like `mongodb-memory-server` for integration tests. Test happy paths, error cases, auth middleware, and validation. Aim for route + service layer tests; end-to-end tests for critical flows.

Question 47: What is WebSocket and when do you need it with Node.js?

Answer: WebSockets provide full-duplex, persistent connections between client and server — unlike HTTP request-response. Use for real-time features: chat, live notifications, collaborative editing, gaming. Libraries: `socket.io` (fallbacks, rooms) or `ws` (minimal). Express handles REST; add a WebSocket server on the same HTTP server or separate port. WebSockets do not replace REST for standard CRUD operations.

Question 48: What is the difference between monolith and microservices with Node.js?

Answer: A monolith is one Node.js app handling all features — simpler to develop and deploy, good for startups. Microservices split features into independent services (user service, order service) communicating via HTTP or message queues — better isolation and scaling but more operational complexity. Start monolithic; extract microservices when teams or scaling demands it. Node.js suits both — lightweight and fast to spin up services.

Question 49: What are common Node.js security best practices?

Answer: (1) Validate all input. (2) Use helmet, HTTPS, HTTP-only cookies. (3) Hash passwords with bcrypt. (4) Keep dependencies updated (`npm audit`). (5) Never expose stack traces. (6) Rate limit auth endpoints. (7) Use parameterized queries (prevent injection). (8) Store secrets in environment variables. (9) Principle of least privilege for database users. (10) Enable Cross-Origin Resource Sharing only for trusted origins.

Question 50: How would you explain your Node.js backend in an interview?

Answer: Cover: (1) Purpose – what the API serves (e-commerce, social app). (2) Stack – Express, MongoDB/Mongoose, JWT auth. (3) Architecture – folder

structure (routes, controllers, services, models), middleware pipeline. (4) Key decisions – why REST, how you handle errors and validation, caching if any. (5) Scale considerations – connection pooling, rate limiting, logging. (6) One challenge you solved – e.g., fixing N+1 queries, securing endpoints, or handling file uploads. Be specific with endpoints and tradeoffs.

Questions 51-100 (top product companies)

Question 51: How do you structure a production Express app?

Answer: routes → controllers → services → repositories. No business logic in routers. Adobe / Razorpay.

Question 52: What is helmet and why do product companies want it?

Answer: Secure headers: CSP, HSTS, frame guard. Default first line of HTTP hardening. Amazon / Flipkart.

Question 53: How do you rotate JSON Web Token refresh tokens?

Answer: One-time refresh, store hash, detect reuse → revoke family. Auth PDF. Amazon / PhonePe.

Question 54: What is the difference between 401 and 403?

Answer: 401: not authenticated. 403: authenticated but not allowed. Object-level checks. All.

Question 55: How do you handle file uploads safely?

Answer: Size limit, mime allow-list, store outside web root / S3, virus scan if needed, unique

names. Amazon / Flipkart.

Question 56: Explain Zod at the boundary.

Answer: Parse req.body once; never trust query strings. Fail 400 with field errors. Stripe-like APIs.

Question 57: How do you do pagination that stays stable when new rows insert?

Answer: Cursor (`id` / `created_at`) not `OFFSET` . Uber / Twitter-style feeds.

Question 58: What is a circuit breaker in a Node service?

Answer: Stop calling a dying dependency after N failures; half-open probe. Uber / Amazon.

Question 59: How do you keep secrets out of logs and git?

Answer: Env + secret manager, redact tokens, git-secrets / scanning. All.

Question 60: process.nextTick vs setImmediate vs setTimeout(0)?

Answer: nextTick before Promises (Node); setImmediate in check phase; setTimeout in timers. Amazon Node.

Question 61: How do you test an Express route?

Answer: Supertest against the app, mock the repository, assert status and body. Phase 4. Adobe.

Question 62: What is graceful degrade when Redis is down?

Answer: Read from DB, hide non-critical features, alert. Do not 500 the whole site. Uber / Flipkart.

Question 63: How do you implement idempotent PUT vs POST?

Answer: PUT same URL same body = same resource. POST create needs Idempotency-Key. Stripe / Razorpay.

Question 64: What is the N+1 in an ORM and the fix?

Answer: Include/join or DataLoader. Log queries in staging. Microsoft / Uber.

Question 65: How do you version a public API?

Answer: URL `/v1` or header. Never break v1; add v2. Adobe / Stripe.

Question 66: What headers do you send for JSON errors?

Answer: `Content-Type: application/json`, stable `{ error, code, requestId }`. All.

Question 67: How do you prevent SSRF when fetching a user URL?

Answer: Allow-list schemes/hosts, block link-local/metadata IPs. Amazon / Google.

Question 68: Explain clustering in Node and its limit.

Answer: One process per core sharing a port. Still one thread per process for JS. CPU work needs workers. Amazon.

Question 69: How do you run migrations in CI/CD?

Answer: Migrate in the release job before traffic switches; backward-compatible steps. Adobe / Amazon.

Question 70: What is a poison message in a queue?

Answer: A message that always fails. After N retries → dead letter + alert. Uber / Amazon.

Question 71: How do you implement search on Postgres cheaply?

Answer: `pg_trgm` / `tsvector` for modest scale; OpenSearch when it outgrows. Flipkart / Adobe.

Question 72: Cookie flags that must be set for auth.

Answer: HttpOnly, Secure, SameSite=Lax or Strict, short max-age. All.

Question 73: How do you do fan-out of emails without blocking the request?

Answer: Insert row + enqueue; 202 Accepted. PDF #10 notifications. Flipkart / Uber.

Question 74: What is request-id and why inject it?

Answer: One id in logs, traces, and the error response. Debug production. Amazon / Google.

Question 75: How do you handle multi-tenant data isolation?

Answer: `tenant_id` on every row + forced WHERE in the repository. Salesforce / Adobe.

Question 76: Explain Prisma vs raw SQL tradeoff.

Answer: Prisma for CRUD speed; raw for ugly queries and EXPLAIN. Microsoft / Uber.

Question 77: How do you cap JSON body size?

Answer: `express.json({ limit: "100kb" })`. Prevent memory bombs. Amazon.

Question 78: What is a health check vs a readiness check?

Answer: Liveness: process up. Readiness: DB pool ok. Kubernetes uses both. Amazon / Adobe.

Question 79: How do you implement feature flags?

Answer: Service like LaunchDarkly or a table; default safe; no deploy to toggle. Meta / Uber.

Question 80: What is the risk of `eval` or `new Function` on the server?

Answer: Remote code execution. Never on user input. All.

Question 81: How do you stream a CSV export?

Answer: SQL cursor + stream to response; do not load all rows. Microsoft / Adobe.

Question 82: Explain eventual consistency in your own API.

Answer: After write, a read replica may lag — read-your-writes from primary. Meta / Uber.

Question 83: How do you protect webhooks?

Answer: Signature (HMAC), timestamp window, idempotency on event id. Stripe / Razorpay / PhonePe.

Question 84: What is the difference between horizontal and vertical scaling for Node?

Answer: Vertical: bigger box. Horizontal: more processes/replicas behind a load balancer. All HLD.

Question 85: How do you do blue-green or rolling deploys?

Answer: New replicas healthy, then shift traffic; keep old for rollback. Amazon / Adobe.

Question 86: What belongs in middleware vs a service?

Answer: Middleware: auth, request-id, language. Service: business rules. Razorpay.

Question 87: How do you handle timezone in APIs?

Answer: Store UTC; display in client locale. Meeting Rooms problems. Uber / Google Calendar-style.

Question 88: What is a leaked promise in a request handler?

Answer: You started async work and responded without waiting or tracking — double send or lost error. Amazon.

Question 89: How do you implement per-user rate limits in Redis?

Answer: INCR + EXPIRE or token bucket Lua. Key = userId. Uber / Amazon.

Question 90: What is OWASP broken access control in a MERN app?

Answer: User A fetches user B's order by changing id. Check ownership every time. All.

Question 91: How do you shut down WebSocket servers gracefully?

Answer: Stop upgrades, close sockets with a code, drain. Uber / Meta chat.

Question 92: Explain exactly-once vs at-least-once for your queue.

Answer: Most Node queues are at-least-once; make handlers idempotent. Amazon / Uber.

Question 93: How do you store passwords?

Answer: bcrypt/argon2, unique salt, never log, rotate on leak. All.

Question 94: What is a BFF and when do you want one?

Answer: Backend-for-frontend aggregates for a specific client. Atlassian / Meta FE.

Question 95: How do you detect a memory leak in Node?

Answer: heap snapshots, rising RSS, `--inspect`. Large caches without TTL. Amazon / Adobe.

Question 96: What is the difference between PUT and PATCH?

Answer: PUT replaces; PATCH partial. Idempotency differs. Stripe-style APIs.

Question 97: How do you implement search-as-you-type on the server?

Answer: Debounce is client; server: prefix index, limit 10, cache hot prefixes. Typeahead HLD. Google / Uber.

Question 98: What logs do you never write?

Answer: Passwords, tokens, card numbers, session ids. PCI / privacy. All.

Question 99: How do you write a postmortem for a 5xx spike?

Answer: Timeline, impact, root cause, blast radius, action items with owners. Amazon LP.

Question 100: Walk through TaskFlow's API like a hostile interviewer.

Answer: Auth, models, sockets, failure modes, what you would shard. Phase 8. All.